OpenSearch Cluster

Kurulum Dokümanı (Ubuntu 24.04)

5 node OpenSearch, yedekli Dashboards, HAProxy ve keepalived

Bilgi	Değer
Ürün	OpenSearch ve OpenSearch Dashboards 3.9.0
İşletim sistemi	Ubuntu 24.04 LTS (24.04.5 ile test edildi)
Doküman sürümü	1.0
Tarih	2 Kasım 2026

İçindekiler

1. Genel bakış
2. Ön koşullar
3. Kurulum öncesi hazırlık
4. Kurulum adımları
5. Kurulumu doğrulama
6. Güvenlik ve TLS
7. Kurulumdan sonra
8. Sorun giderme
Ek A. Test kapsamı
Ek B. Docker'da deneme

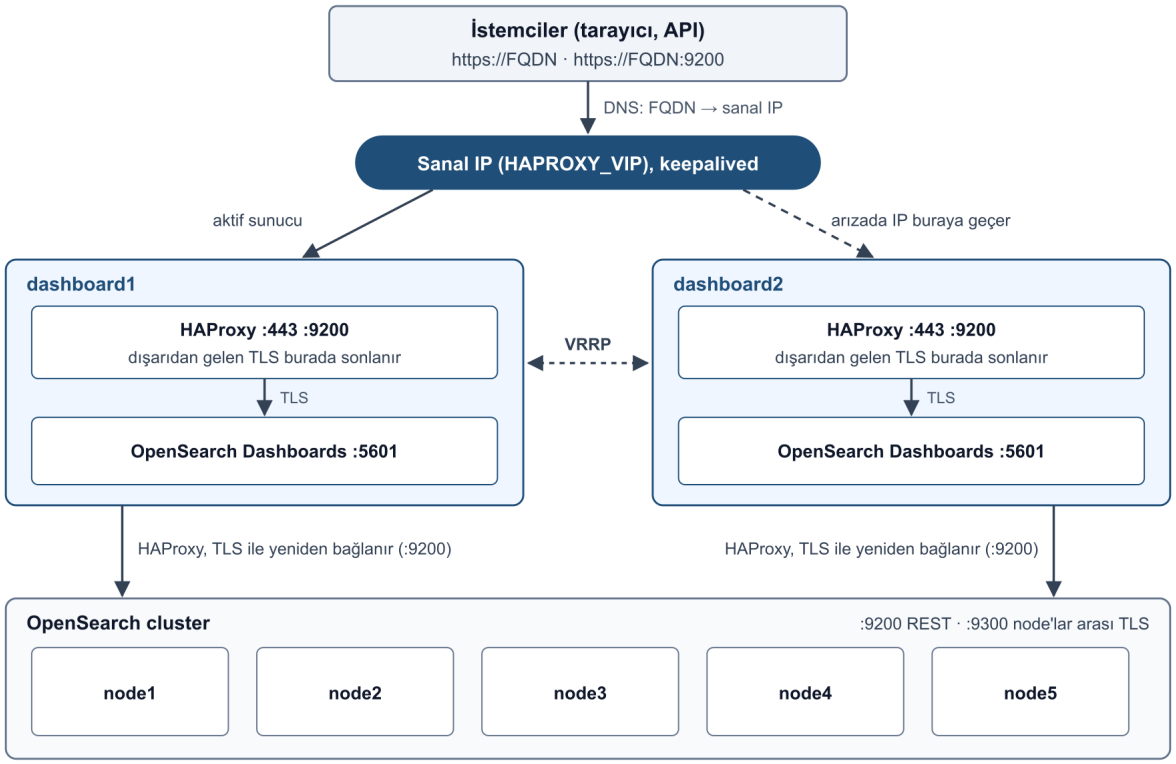
1. Genel bakış

Bu doküman, OpenSearch 3.9 cluster'ının kurulum script'leriyle Ubuntu 24.04 LTS sunuculara nasıl kurulacağını adım adım anlatır. Kurulumu yapacak, Linux'ta root yetkisiyle çalışan sistem yöneticileri için yazılmıştır.

Kurulum sonunda şu yapı çalışır:

- **OpenSearch:** 5 node'lu cluster. Tüm node'lar cluster manager adayıdır, veri tutar ve ingest yapar.
- **OpenSearch Dashboards:** iki ayrı sunucuda (dashboard1, dashboard2), aktif-aktif ve ortak oturum anahtarıyla.
- **HAProxy ve keepalived:** iki dashboard sunucusunda. Dışarıdan tek adres ve sanal IP sağlar.
- **Sertifikalar:** demo ayarları kullanılmaz; sertifikalar kurulum sırasında üretilen kendi CA'mızla imzalanır.

1.1 Mimari



İki HAProxy de iki Dashboards'a ve 5 node'un hepsine bağlanır. Sanal IP'nin hangi sunucuda olduğu arka uçların kullanımını değiştirmez.

Adres	Port	Hedef
`https://<DASHBOARDS_FQDN>`	443	OpenSearch Dashboards (dashboard1, dashboard2)
`https://<DASHBOARDS_FQDN>:9200`	9200	OpenSearch REST API (node1 ... node5)

1.2 Sunucular

Sunucu	Görev	Portlar
node1 ... node5	OpenSearch: `cluster_manager`, `data`, `ingest`	9200, 9300
dashboard1, dashboard2	OpenSearch Dashboards, HAProxy, keepalived	5601, 443, 9200, VRRP

Cluster'ın çalışması için en az 3 node ayakta olmalıdır; aynı anda 2 node kaybı tolere edilir.

1.3 Kapsam dışı

- DNS kaydının açılması (`DASHBOARDS_FQDN` -> `HAPROXY_VIP`).
- Çalışan bir cluster'a node eklemek, Dashboards'u başka sunucuya taşımak ve sürüm yükseltmek. Script'ler sıfırdan kurulum içindir.
- Donanım boyutlandırması. Heap varsayılan olarak RAM'in yarısıdır (en fazla 31 GB).

2. Ön koşullar

2.1 Sunucular

- 7 adet Ubuntu 24.04 LTS sunucu (x86_64 ya da aarch64) ve root yetkisi. Komutlar root olarak çalıştırılır (`sudo -i`).
- Node'larda veri için `/data`. Ayrı bir disk önerilir; zorunlu kılmak için `REQUIRE_DATA_MOUNT="yes"` kullanılır.
- Saat senkronu (systemd-timesyncd ya da chrony). Node saatleri aynı olmalıdır.
- Dashboard sunucularında Ubuntu deposuna (ya da yerel aynaya) erişim: `haproxy` ve `keepalived` `apt` ile kurulur. Node'larda depo erişimi gerekmez.
- Eksik sistem komutları `hazirlik.sh` tarafından en başta listelenir. Örneğin ham Ubuntu 24.04 konteyner imajında `apt-get install curl iproute2 systemd openssl gnupg ca-certificates` gerekmiştir.

2.2 Önceden belirlenecek değerler

Değer	Açıklama
7 sunucu IP'si	5 node ve 2 dashboard sunucusu; hepsi birbirinden farklı IPv4 adresleri.
`DASHBOARDS_FQDN`	Dışarıdan kullanılan tek ad (ör. `opensearch.sirket.local`). Sunucu adlarından farklı olmalı.
`HAPROXY_VIP`	Dashboard sunucularıyla aynı subnet'te, kullanılmayan bir IP. Başka bir sisteme atanmamalı.
`admin` ve `kibanaserver` şifreleri	En az 8 karakter; birbirinden farklı olması önerilir. Kurulum sırasında sorulur, dosyaya yazılmaz.
`CERT_SUBJECT`	Sertifika kurum bilgisi (ör. `/C=TR/O=Sirket/OU=IT`). Türkçe karakter kullanılmaz.
Erişim kısıtları	443 ve 9200'e erişecek CIDR'lar (bkz. 3.1).

2.3 Ağ erişimleri

ufw etkinse sunucu kuralları script'lerce eklenir. Aradaki ağ firewall'larında aşağıdaki erişimler ayrıca açık olmalıdır:

Kaynak	Hedef	Port / protokol	Amaç
node1 ... node5	node1 ... node5	9200, 9300/tcp	Cluster iletişimi
dashboard1, dashboard2	node1 ... node5	9200/tcp	Dashboards ve HAProxy
dashboard1, dashboard2	dashboard1, dashboard2	5601/tcp, VRRP (IP protokol 112)	HAProxy, sanal IP
Kullanıcılar, uygulamalar	`HAPROXY_VIP`	443, 9200/tcp	Dışarıdan erişim
node1	Diğer 6 sunucu	22/tcp	Paket dağıtımı (scp kullanılacaksa)

> **Dikkat:** Sanal IP için ağın VRRP'ye ve IP'nin sunucular arasında taşınmasına izin vermesi gerekir. VMware'de port grubu ayarları, bulutta ağ politikası bunu engelleyebilir. Kurulumdan önce ağ ekibiyle teyit edin.

2.4 Paketler

Aşağıdaki dosyalar indirilip her sunucuda `/root/opensearch-deb` altına konur. aarch64 sunucularda dosya adlarında `x64` yerine `arm64` kullanılır.

Dosya	Konacağı sunucu
`opensearch-3.9.0-linux-x64.deb` ve `.sig`	node1 ... node5
`opensearch-dashboards-3.9.0-linux-x64.deb` ve `.sig`	dashboard1, dashboard2
`opensearch-release.pgp`	7 sunucu

```
B=https://artifacts.opensearch.org/releases/bundle
curl -fLO $B/opensearch/3.9.0/opensearch-3.9.0-linux-x64.deb
curl -fLO $B/opensearch/3.9.0/opensearch-3.9.0-linux-x64.deb.sig
curl -fLO $B/opensearch-dashboards/3.9.0/opensearch-dashboards-3.9.0-linux-x64.deb
curl -fLO $B/opensearch-dashboards/3.9.0/opensearch-dashboards-3.9.0-linux-x64.deb.sig
curl -fLO https://artifacts.opensearch.org/publickeys/opensearch-release.pgp
```

DEB paketleri kurulmadan önce `.sig` dosyasıyla doğrulanır. Bağımlılıkları olmadığı için internetsiz sunucuya da kurulabilirler.

3. Kurulum öncesi hazırlık

3.1 config.env dosyası

Tüm ayarlar `config.env` dosyasındadır. Aşağıdaki değerler girilmeden script'ler çalışmaz (adresler örnektir):

```
NODE1_IP="10.0.0.11"
NODE2_IP="10.0.0.12"
NODE3_IP="10.0.0.13"
NODE4_IP="10.0.0.14"
NODE5_IP="10.0.0.15"
DASHBOARD1_IP="10.0.0.21"
DASHBOARD2_IP="10.0.0.22"
DASHBOARDS_FQDN="opensearch.sirket.local"
HAPROXY_VIP="10.0.0.20"
```

Gerektiğinde değiştirilecek diğer ayarlar:

Ayar	Varsayılan	Açıklama
`CLUSTER_NAME`	`opensearch-cluster`	Cluster adı; ana log dosyasının adı da budur.
`CERT_SUBJECT`	`/C=TR/O=Sirket/OU=IT`	Sertifika kurum bilgisi. Türkçe karakter kullanmayın; DN eşleşmesini bozar.
`HEAP_SIZE`	`auto`	RAM'in yarısı, en fazla 31 GB. Elle: `8g`.
`DATA_DIR`, `LOG_DIR`	`/data/opensearch`, `/data/logs`	Veri ve log dizinleri.
`DASHBOARDS_ALLOWED_SOURCES`	boş	443'e erişecek CIDR'lar. **Boşsa herkese açık.**
`OPENSEARCH_LB_ALLOWED_SOURCES`	boş	HAProxy'nin 9200 portuna erişecek CIDR'lar. **Boşsa herkese açık.**
`HTTP_ALLOWED_SOURCES`	boş	Node'ların 9200 portuna doğrudan erişecek ek CIDR'lar.
`HAPROXY_VRID`	`51`	VRRP küme numarası (1-255). Aynı ağdaki başka keepalived kümeleriyle çakışmamalı.
`HAPROXY_DASHBOARDS_PORT`	`443`	Dashboards'un dış portu.

Ayar	Varsayılan	Açıklama
`HAPROXY_OPENSEARCH_PORT`	`9200`	OpenSearch'ün dış portu.
`DISTRIBUTE_WITH_SCP`	`yes`	node1 paketleri `root@<sunucu>` hedefine scp ile göndermeyi dener (bkz. Adım 2).
`DASHBOARDS_CERT_MODE`	`auto`	Kurum sertifikası ya da üretilen sertifika (bkz. 6.2).

> **Dikkat:** Erişim kısıtları ufw etkinse uygulanır. Üretim ortamında `DASHBOARDS_ALLOWED_SOURCES` ve `OPENSEARCH_LB_ALLOWED_SOURCES` değerlerini doldurmanız önerilir. Giriş yine şifre ister, ama portlar varsayılan olarak herkese açıktır.

3.2 Script'lerin dağıtımı

Script dizinini 7 sunucuya aynı yere kopyalayın (ör. `/root/opensearch-kurulum`). `config.env` 7 sunucuda aynı olmalıdır; node ve dashboard script'leri bunu kontrol eder, farklıysa durur.

Dosya	Çalıştığı yer	Görevi
`config.env`	7 sunucu	Tüm ayarlar.
`lib.sh`	(doğrudan çalıştırılmaz)	Ortak fonksiyonlar.
`hazirlik.sh`	7 sunucu	Paketler, `/etc/hosts`, kernel ayarları, dizinler ve firewall kuralları.
`node1.sh`	node1	Sertifikalar, şifreler, paketler, node1 OpenSearch.
`node2.sh` ... `node5.sh`	node2 ... node5	Node'u kurar ve cluster'a katar.
`dashboard1.sh`, `dashboard2.sh`	dashboard1, dashboard2	Dashboards, HAProxy ve keepalived kurulumu, test.
`opensearch-dashboard-test.sh`	dashboard1, dashboard2	Uçtan uca test; `dashboardN.sh` sonunda kendiliğinden çalışır.

Her script yanlış sunucuda çalıştırılırsa durur. Sunucu, IP adresinden tanınır.

3.3 Kurum sertifikası (isteğe bağlı)

Kurum sertifikası kullanılacaksa, her dashboard sunucusunda `/root/dashboards-certs/` altına `ssl.crt`, `ssl.key` ve `ca.crt` koyun. Sertifika, SAN alanında `DASHBOARDS_FQDN` değerini içermelidir. Konmazsa cluster'ın root CA'sıyla imzalı sertifika kullanılır.

4. Kurulum adımları

Sıra önemlidir. Node'lar paralel kurulabilir, Dashboards sunucuları ise sırayla kurulmalıdır.

Adım	Sunucu	Komut	Başarı işareti
1	7 sunucu	`./hazirlik.sh`	`... hazırlığı tamamlandı`
2	node1	`./node1.sh`	`Cluster'ın oluşması bekleniyor`
3	node2 ... node5	`./nodeN.sh`	`nodeN tamamlandı`
4	node1	(bekleyen `node1.sh`)	`Cluster hazır: 5 node, durum green`
5	dashboard1	`./dashboard1.sh`	`Sonuç: 12 başarılı, 0 başarısız`
6	dashboard2	`./dashboard2.sh`	`Sonuç: 14 başarılı, 0 başarısız`
7	DNS	FQDN -> sanal IP	Adres tarayıcıda açılır

Adım 1: Ortak hazırlık (7 sunucu)

Sıra fark etmez. Önce yalnızca kontrol yapın, sonra kurun:

```
cd /root/opensearch-kurulum
./hazirlik.sh --check      # yalnızca kontrol, değişiklik yapmaz
./hazirlik.sh
```

Script; `/etc/hosts`, swap ve `vm.max_map_count` ayarlarını yapar, DEB paketlerini imza doğrulamasıyla kurar. Dashboard sunucularına HAProxy ve keepalived'ı da kurar. Servisler bu aşamada başlatılmaz; kurulunca kendiliğinden başlayan HAProxy ve keepalived da kapatılır.

Ubuntu'nun sunucu adını yazdığı `127.0.1.1` satırı yorum satırı yapılır; aksi halde ad gerçek IP yerine bu adrese çözülür. `unattended-upgrades` dpkg kilidini tutuyorsa kurulum bitmesini 5 dakikaya kadar bekler.

> **Not:** Script ufw'yi kendisi etkinleştirmez. ufw sonradan etkinleştirilecekse önce SSH'a izin verin, etkinleştirdikten sonra `./hazirlik.sh`'i tekrar çalıştırın.

Adım 2: node1

```
./node1.sh                 # admin ve kibanaserver şifrelerini sorar
```

Script tüm sertifikaları üretir, diğer 6 sunucunun paketlerini hazırlar ve scp ile gönderir, node1'de OpenSearch'ü başlatır. `Cluster'ın oluşması bekleniyor` yazdığında bu terminali açık bırakıp 3. adıma geçin (en fazla 30 dakika bekler).

Ubuntu'da root ile SSH girişi varsayılan olarak kapalıdır, bu yüzden otomatik gönderim çoğu sunucuda başarısız olur ve script uyarı verir. Ekrana yazdığı `root@` komutu da aynı nedenle çalışmaz. Bu normaldir; paketleri sudo yetkili bir kullanıcıyla elle kopyalayın ve hedef sunucuda `/root` altına taşıyın:

```
# node1'de, root olarak (... node3, node4, node5, dashboard1, dashboard2)
scp /root/opensearch-bundle/opensearch-bundle-node2.tar.gz <kullanici>@node2:/tmp/

# Hedef sunucuda
sudo install -o root -g root -m 600 /tmp/opensearch-bundle-node2.tar.gz /root/
rm -f /tmp/opensearch-bundle-node2.tar.gz
```

Adım 3: node2 ... node5

Her node'da kendi script'ini çalıştırın (sıra fark etmez):

```
./node2.sh                 # node2'de
./node3.sh                 # node3'te
./node4.sh                 # node4'te
./node5.sh                 # node5'te
```

5 cluster manager adayından 3'ü bir araya gelene kadar cluster oluşmaz; ilk başlatılan node'lar bu sırada bekler. Bu normaldir.

Adım 4: Cluster'ın tamamlanması

node1'deki script, 5 node katılıp cluster `green` olunca kendiliğinden biter ve şu satırı yazar:

```
[OK] Cluster hazır: 5 node, durum green
```

Adım 5: dashboard1

```
./dashboard1.sh            # kibanaserver ve admin şifrelerini sorar
```

Script Dashboards'u başlatır, HAProxy ve keepalived'ı yapılandırır, sanal IP'nin yanıt vermesini bekler ve uçtan uca testi çalıştırır. Beklenen sonuç:

```
Sonuç: 12 başarılı, 0 başarısız
```

> **Dikkat:** dashboard1 bitmeden dashboard2'yi başlatmayın. İlk açılışta `.kibana` index'i oluşturulur; iki sunucu aynı anda başlarsa yarım kalan migration elle temizlik gerektirebilir.

Adım 6: dashboard2

```
./dashboard2.sh
```

Beklenen sonuç aşağıdadır. Fazladan 2 test, diğer Dashboards'a erişimi ve açılan oturumun iki sunucuda da geçerli olduğunu doğrular.

```
Sonuç: 14 başarılı, 0 başarısız
```

Adım 7: DNS kaydı

`DASHBOARDS_FQDN` -> `HAPROXY_VIP` kaydını açtırın. Kayıt hazır olmadan denemek için istemcinin hosts dosyasına şu satır eklenebilir:

```
<HAPROXY_VIP> <DASHBOARDS_FQDN>
```

Bir adım yarıda kalırsa

Tüm script'ler tekrar çalıştırılabilir; mevcut CA, sertifikalar, `internal_users.yml` ve oturum anahtarı yeniden kullanılır. Sebebi düzeltip aynı script'i tekrar çalıştırın. `node1.sh` aşamalı da çalıştırılabilir:

Komut	Ne yapar
`./node1.sh certs`	Sertifikalar, şifre hash'leri, oturum anahtarı, paketler ve dağıtım.
`./node1.sh opensearch`	node1'i başlatır, cluster'ın green olmasını bekler.
`./dashboard1.sh <paket>`	Paket varsayılan yerde değilse.
`./dashboard1.sh test`	Yalnızca test.

> **Not:** Otomasyon için şifreler `ADMIN_PASSWORD` ve `KIBANASERVER_PASSWORD` ortam değişkenleriyle verilebilir. Şifreler komut satırına yazılmaz, `ps` çıktısında görünmez.

5. Kurulumu doğrulama

5.1 Kontroller

Aşağıdaki komutlarda root CA, node1'deki `/root/opensearch-certs/root-ca.pem` dosyasıdır.

```
# Servisler
systemctl is-active opensearch                                  # node'larda
systemctl is-active opensearch-dashboards haproxy keepalived    # dashboard'larda

# Sanal IP'nin bulunduğu sunucu (yalnızca birinde çıktı vermeli)
ip -4 -o addr show | grep '<HAPROXY_VIP>/'

# Cluster, HAProxy üzerinden
curl --cacert root-ca.pem -u admin 'https://<DASHBOARDS_FQDN>:9200/_cat/nodes?v'
curl --cacert root-ca.pem -u admin 'https://<DASHBOARDS_FQDN>:9200/_cluster/health?pretty'

# Uçtan uca test (dashboard sunucusunda)
KIBANASERVER_PASSWORD='<şifre>' ./dashboard1.sh test
```

İki dashboard da çalışırken test `14 başarılı` yazmalıdır. `KIBANASERVER_PASSWORD` verilmezse kibanaserver testi atlanır ve sonuç bir eksik olur. Son olarak tarayıcıdan `https://<DASHBOARDS_FQDN>` adresine `admin` ile giriş yapın.

5.2 Arıza denemesi (önerilir)

Sanal IP'nin gerçek ağda geçtiğini canlıya almadan önce doğrulayın:

1. Sanal IP'nin bulunduğu sunucuda `systemctl stop haproxy` çalıştırın.
2. Birkaç saniye içinde IP'nin diğer sunucuya geçtiğini ve `https://<DASHBOARDS_FQDN>` adresinin açıldığını kontrol edin.

3. Açık tarayıcı oturumunun düşmediğini kontrol edin.
4. `systemctl start haproxy` ile servisi geri açın. IP bilerek geri dönmez (`nopreempt`); bu, bağlantıların ikinci kez kesilmesini önler.

6. Güvenlik ve TLS

6.1 TLS akışı

Bağlantı	Sertifika	Doğrulama
İstemci -> HAProxy (443, 9200)	Dashboards sertifikası (`/etc/haproxy/certs/site.pem`)	İstemci `DASHBOARDS_FQDN` ile doğrular
HAProxy -> Dashboards (5601)	Dashboards sertifikası	HAProxy, CA ve `DASHBOARDS_FQDN` ile doğrular
HAProxy -> node (9200)	Node sertifikası	HAProxy, root CA ve node adıyla doğrular
Dashboards -> node (9200)	Node sertifikası	Root CA ve node adıyla doğrulanır (`full`)
Node <-> node (9300)	Node sertifikası	Root CA ve `nodes_dn` listesi

Bu yapının bilinmesi gereken sonuçları:

- `admin.pem` **HAProxy üzerinden çalışmaz.** TLS HAProxy'de sonlandığı için istemci sertifikası node'a ulaşmaz ve istek 401 alır. Admin sertifikası gereken işlemler (`securityadmin.sh`, admin API) doğrudan bir node'a yapılır: `https://node1:9200`.
- **HAProxy, node sertifikasını kullanmaz.** `nodes_dn` listesindeki bir sertifikanın key'i, sahibini cluster node'u gibi gösterir. HAProxy, aynı CA'dan imzalı ve yalnızca `serverAuth` yetkili Dashboards sertifikasını kullanır.
- **5601 kullanıcılara kapalıdır.** Yalnızca iki dashboard sunucusuna açılır; kullanıcılar 443'ü kullanır.
- **İstemci IP'si** `X-Forwarded-For` başlığında iletilir. Node'lar ve Dashboards bağlantıyı HAProxy sunucusunun IP'sinden görür.
- `site.pem` **şifresiz key içerir** (izin 600), çünkü HAProxy şifreli key okuyamaz.

6.2 Dashboards sertifikası

`DASHBOARDS_CERT_MODE` ile seçilir. HAProxy dışarıya bu sertifikayı gösterir.

Mod	Davranış
`auto` (varsayılan)	Dashboard sunucusunda `/root/dashboards-certs` içinde `ssl.crt` ve `ssl.key` varsa onları, yoksa node1'in ürettiği sertifikayı kullanır.
`provided`	Kurum sertifikası zorunludur: `ssl.crt`, `ssl.key`, `ca.crt`. node1 Dashboards sertifikası üretmez.
`self`	Cluster'ın root CA'sıyla imzalı sertifika. SAN: `DASHBOARDS_FQDN`, `HAPROXY_VIP`, sunucunun adı ve IP'si (+ `DASHBOARDS_EXTRA_SAN`).

Kurum sertifikası için:

- SAN alanı `DASHBOARDS_FQDN` değerini içermelidir; içermiyorsa script durur.
- Ara CA varsa `ssl.crt` önce sunucu, sonra ara CA sertifikasını içermelidir (fullchain).
- Key şifreliyse şifre sorulur ya da `DASHBOARDS_KEY_PASSPHRASE` ile verilir.

6.3 ufw ve AppArmor

Durum	Davranış
ufw etkin (node)	9300 yalnızca 5 node'a; 9200 node'lara, dashboard sunucularına ve `HTTP_ALLOWED_SOURCES`'a açılır.
ufw etkin (dashboard)	5601 ve VRRP iki dashboard sunucusuna; 443 `DASHBOARDS_ALLOWED_SOURCES`'a, 9200 `OPENSEARCH_LB_ALLOWED_SOURCES`'a açılır (boşsa herkese; doluysa iki dashboard IP'si de eklenir).
ufw etkin değil ya da kurulu değil	Kural eklenmez. Script ufw'yi etkinleştirmez.
firewalld ya da nftables aktif	Uyarı verilir, kural eklenmez; kuralları kendiniz ekleyin.
AppArmor	Paketler profil getirmediği için ayrı bir adım yoktur. Kendi profillerinizi yüklediyseniz bu servisleri kontrol edin.

7. Kurulumdan sonra

7.1 Yapılacaklar

1. `/root/opensearch-certs` dizinini (root CA key'i içerir) güvenli bir yere yedekleyip node1'den kaldırın. Yeni sertifika üretmek için bu key gerekir.
2. Paketleri silin; private key ve oturum anahtarı içerirler: node1'de `/root/opensearch-bundle/`, diğer sunucularda `/root/opensearch-bundle-*.tar.gz`.
3. `self` modda tarayıcı uyarısı olmaması için `root-ca.pem` dosyasını istemcilere güvenilir kök sertifika olarak dağıtın.
4. Sertifika bitiş tarihlerini (varsayılan 730 gün) takvime ekleyin.
5. Erişim kısıtlarını gözden geçirin (bkz. 3.1).

```
openssl x509 -enddate -noout -in /etc/opensearch/node1.pem                  # node'larda
openssl x509 -enddate -noout -in /etc/opensearch-dashboards/certs/ssl.crt   # dashboard
```

7.2 Bakım işlemleri

İşlem	Nasıl yapılır
Şifre değişikliği	Dashboards: Security > Internal users ya da `securityadmin.sh`. `internal_users.yml` yalnızca cluster ilk kurulurken okunur.
`kibanaserver` şifresi değişti	İki dashboard sunucusunda `./dashboardN.sh` tekrar çalıştırılır.
Dashboards sertifikası yenilendi	`./dashboardN.sh` tekrar çalıştırılır; `site.pem` ve HAProxy güncellenir.
IP, FQDN, VIP ya da `CERT_SUBJECT` değişti	İlgili sertifika yeniden üretilir: `./node1.sh certs`, paketleri dağıtıp ilgili script'leri tekrar çalıştırın.
Yeni istemci subnet'i	`config.env`'yi 7 sunucuda güncelleyip ilgili sunucularda `./hazirlik.sh` çalıştırın.

> **Dikkat:** `./dashboardN.sh` node1'in paketini ister. Paket silindiyse node1'de `./node1.sh certs` ile yeniden üretilir, ama önce `/root/opensearch-certs` yedekten geri konmalıdır. Dizin yoksa script **yeni bir root CA üretir** ve paketler çalışan cluster'la uyumsuz olur.

Güvenlik dosyalarındaki değişiklikler (`/etc/opensearch/opensearch-security/`) node1'de `securityadmin.sh` ile uygulanır. Bu komut dizindeki tüm dosyaları yükler ve Dashboards ya da REST API ile yapılmış değişikliklerin üzerine yazar; önce yedek alın:

```
export OPENSEARCH_JAVA_HOME=/usr/share/opensearch/jdk
SA=/usr/share/opensearch/plugins/opensearch-security/tools/securityadmin.sh
C=/etc/opensearch
CERTS="-cacert $C/root-ca.pem -cert $C/admin.pem -key $C/admin-key.pem"

# Yedek
$SA -backup /root/security-backup-$(date +%F) $CERTS -icl -nhnv
# Değişiklikleri uygula
$SA -cd $C/opensearch-security/ $CERTS -icl -nhnv
```

8. Sorun giderme

8.1 Belirtiler

Belirti	Olası sebep	Ne yapılmalı
`hazirlik.sh` `Eksik komut(lar)` diyor	Minimal kurulumda paket eksik	Listelenen komutların paketlerini kurun. `ip` ve `ss` `iproute2`, `cmp` ve `diff` `diffutils`, `gpg` `gnupg` paketindedir.
hazirlik.sh `node1 -> ... (beklenen ...)` diyor	`/etc/hosts`'ta ada farklı IP veren başka satır var (`127.0.1.1` satırı otomatik düzeltilir)	Çakışan satırı düzeltin. cloud-init kullanılıyorsa kayıtları `hosts.debian.tmpl` şablonuna da ekleyin.
Paket gönderilirken `scp ile kopyalanamadı`	Ubuntu'da root SSH girişi varsayılan olarak kapalı	Adım 2'deki gibi elle kopyalayın.
node1.sh `cluster henüz oluşmadı` diyerek bekliyor	5 node'dan 3'ü bir araya gelmedi	node2-5 script'lerini çalıştırın. Script 2 dakikada bir teşhis basar: 9300 erişimi ve TLS hataları.
node1.sh `cluster: 4/5 node` diyerek bekliyor	Bir node katılmadı	O node'da script'i çalıştırın; logunda TLS ve `nodes_dn` hatalarına bakın.
Logda `nodes_dn` hatası	Sertifika DN'leri listeyle eşleşmiyor	`./node1.sh certs` ile paketleri yeniden üretip dağıtın, node script'lerini tekrar çalıştırın.
`unknown_ca` / `PKIX path building failed`	Node'lar farklı CA'larla imzalı	Sertifikaları yalnızca node1 üretmeli; paketleri yeniden dağıtın.
`config.env ... aynı değil`	Sunuculardaki dosya farklı	node1'deki `config.env`'yi kopyalayın.
OpenSearch açılmıyor, `jna ... failed to map segment`	`/tmp` noexec	`./hazirlik.sh`'i tekrar çalıştırın.
`cluster UUID` uyumsuzluğu	Node tek başına bootstrap olmuş	**Yalnızca yeni kurulumda:** `systemctl stop opensearch`, `rm -rf /data/opensearch/*`, node script'ini tekrar çalıştırın.
Servis `permission denied` ile düşüyor	Dosya sahipliği	`ls -ld /data/opensearch /data/logs`, ardından `./hazirlik.sh`.
`Hiçbir OpenSearch node'unun REST'ine ulaşılamadı`	Cluster kurulmadı ya da 9200 kapalı	node1.sh'in bittiğini ve ağ firewall'unu kontrol edin; node'larda `./hazirlik.sh`'i tekrar çalıştırın.
`<VIP>:443 30 saniyede açılmadı`	VIP subnet dışında ya da kullanımda, ya da VRRP engelli	`journalctl -u keepalived`, `ip -4 addr`. İki sunucu da kendini MASTER sanıyorsa VRRP ulaşmıyordur.
`journalctl -u haproxy`: `Server ... is DOWN`	Sağlık kontrolü başarısız	Servisi ve `ufw status` çıktısını kontrol edin.
HAProxy üzerinden OpenSearch testi başarısız	HAProxy node'a ulaşamıyor ya da sertifikayı doğrulayamıyor	`journalctl -u haproxy`. `/etc/haproxy/certs/opensearch-ca.pem` root CA ile aynı olmalı.

Belirti	Olası sebep	Ne yapılmalı
`admin.pem` ile HAProxy üzerinden 401	TLS HAProxy'de sonlanıyor	Doğrudan node'a bağlanın: `https://node1:9200`.
Dashboards `server is not ready yet`	kibanaserver şifresi, root CA yolu ya da isim çözümleme	`journalctl -u opensearch-dashboards`, `/etc/hosts`.
`Another OpenSearch Dashboards instance appears to be migrating the index`	İki sunucu aynı anda ilk kez başladı	Diğerinin açılmasını bekleyin. Açılmıyorsa ikisini durdurun, yarım `.kibana_N` index'ini silin, önce dashboard1'i başlatın.
`Oturum diğer Dashboards'ta geçersiz`, oturum arada düşüyor	İki sunucuda oturum anahtarı farklı	Aynı `node1.sh certs` çıktısının paketleriyle iki sunucuda da `./dashboardN.sh` çalıştırın.
Tarayıcı sertifika uyarısı veriyor	Farklı ad yazıldı ya da CA güvenilir değil	Ortak adı kullanın, root CA'yı istemcilere dağıtın.
Adres açılmıyor, `./dashboardN.sh test` başarılı	DNS hazır değil ya da istemciden VIP'ye erişim kapalı	`getent hosts <DASHBOARDS_FQDN>` VIP'yi vermeli; ağ firewall'unda 443 ve 9200'ü kontrol edin.

8.2 Teşhis komutları

```
# Cluster neden oluşmuyor
grep "cluster-manager not discovered" /data/logs/<CLUSTER_NAME>.log | tail -2

# TLS hataları
grep -iE "ssl|handshake|certificate|unknown_ca|PKIX" /data/logs/<CLUSTER_NAME>.log \
  | grep -E "WARN|ERROR" | tail -5

# Cluster durumu (node1, admin sertifikasıyla, şifre gerekmez)
curl --cert /etc/opensearch/admin.pem --key /etc/opensearch/admin-key.pem \
  "https://node1:9200/_cat/nodes?v"

# HAProxy arka uç durumu ve sanal IP geçişleri
journalctl -u haproxy -n 30 --no-pager | grep -E " is (UP|DOWN)"
journalctl -u keepalived -n 30 --no-pager | grep -E "MASTER|BACKUP|FAULT"

# Root CA 7 sunucuda aynı olmalı (fingerprint)
openssl x509 -noout -fingerprint -sha256 -in /etc/opensearch/root-ca.pem
openssl x509 -noout -fingerprint -sha256 -in /etc/opensearch-dashboards/root-ca.pem

# Oturum anahtarı iki dashboard sunucusunda aynı olmalı
grep cookie.password /etc/opensearch-dashboards/opensearch_dashboards.yml | sha256sum
```

Ek A. Test kapsamı

Script'ler 2 Kasım 2026'da, Docker üzerinde Ubuntu 24.04.5 LTS + systemd konteynerleriyle, aarch64 mimarisinde ve OpenSearch 3.9.0 (HAProxy 2.8, keepalived 2.2.8) ile uçtan uca çalıştırılmıştır. Süreler Docker ağında ölçülmüştür; gerçek ağda farklı olabilir.

Senaryo	Sonuç
Sıfırdan kurulum (ufw açık ve kapalı)	dashboard1: 12/12, dashboard2: 14/14 test başarılı; cluster green.
Yetkisiz istemciden erişim (ufw açık)	HAProxy 443 ve 9200 açık; 5601, node 9200 ve 9300 kapalı.
Erişim kısıtı tanımlıyken	ufw kuralları doğru yazıldı, iki dashboard IP'si de eklendi.
`127.0.1.1` satırı	Yorum satırı yapıldı; ad gerçek IP'ye çözüldü.
Dış sertifika (443 ve 9200)	Root CA ve ortak adla doğrulandı.
OpenSearch yük dağılımı	20 istek 5 node'a eşit dağıldı.
node3 kapalıyken	20 istekte hata yok; cluster yellow oldu, node geri katılınca green.
Aktif sunucuda HAProxy durunca	Sanal IP diğer sunucuya geçti; 38 istekte 1 hata.

Senaryo	Sonuç
Aktif sunucu kapanınca	Sanal IP geçti; 37 istekte 3 hata (yaklaşık 5 saniye). Açık oturum korundu.
Sunucu geri gelince	Sanal IP geri dönmedi, tek sunucuda kaldı.
Script'leri tekrar çalıştırma, durdurup başlatma	Hatasız; veri ve ufw kuralları korundu.

> **Dikkat:** Şunlar konteyner ortamında denenmediği için **test edilmemiştir** ve gerçek sunucularda ilk kez çalışacaktır: gerçek bir Ubuntu sunucu kurulumu, scp ile paket dağıtımı, gerçek ağda VRRP, x86_64 mimarisi, cloud-init'in `/etc/hosts`'u yeniden yazması, dpkg kilidi tutulurken bekleme ve AppArmor açık çalışma. İlk kurulumda bu adımları yakından izleyin ve 5.2'deki arıza denemesini yapın.

Ek B. Docker'da deneme

`docker/` dizini, script'leri gerçek sunucu olmadan konteynerlerde çalıştırır. Script'ler değiştirilmez; `docker/config.env` ana `config.env`'den üretilir. Yalnızca yerel deneme içindir; en az 8 GB bellek gerekir. `ENABLE_UFW=yes ./up.sh` ilk kurulumdan önce ufw'yi etkinleştirir.

```
cd docker
./up.sh                  # DEB'leri indirir, kurar ya da kaldığı yerden başlatır
docker compose stop      # durdurur, veri korunur
docker compose down -v   # kurulumu ve veriyi siler
```

Dashboards `https://localhost:8443` (`admin`, şifre `docker/.env`), OpenSearch `https://localhost:9200` adresindedir; sertifika `docker/root-ca.pem` ile doğrulanır. Test edilen Mac'te host portu 443 TLS el sıkışmasında bağlantıyı sıfırladığı için localhost tarafında 8443 kullanılır; konteynerlerde HAProxy yine 443'ü dinler. RHEL ortamıyla aynı portları kullandığı için ikisi aynı anda çalıştırılmaz.